CRESCENDO STUDY GUIDE

Beginner-Friendly Project Explanation

A simple, casual guide that explains what the app does, how the code works, and what to say during presentation or Q&A.

PROJECT	MAIN TECH
CRESCENDO-FINALS-APPTECH	React, Vite, React Router
STORAGE	**UPDATED**
Browser localStorage	5/12/2026, 3:15:53 AM

1. Super Quick Project Summary

> **Simple explanation:** Crescendo is a music learning website. A student logs in, picks an instrument, studies lessons, takes quizzes, saves notes, and tracks progress.

Think of Crescendo like a small online music academy. It has lessons for guitar, piano, drums, ukulele, violin, and bass. Each instrument has Beginner, Intermediate, and Advanced levels. The student must pass quizzes to unlock harder levels.

The project is built with React, so the screen is made from components. It uses React Router so the app can have different pages, like Login, Home, Dashboard, Lesson, Quiz, Progress, and Resources. It uses localStorage so progress stays saved in the browser even after refreshing.

Best 30-second explanation

“Our app is called Crescendo. It is a React music-learning platform where students can register or use a demo account, choose an instrument, study lessons, take quizzes, save lesson notes, and track progress. The app uses React Router for navigation, React state for interactive features, and localStorage for saved accounts and progress.”

2. What The App Can Do

- **Login and register:** Students can create an account or use the demo account.
- **Separate progress:** Each account has its own saved progress.
- **Instrument studios:** There are 6 instruments: guitar, piano, drums, ukulele, violin, and bass.
- **Lesson tiers:** Each instrument has Beginner, Intermediate, and Advanced lessons.
- **54 lessons total:** 6 instruments × 3 tiers × 3 lessons.
- **Quizzes:** Every lesson has a quiz. Passing score is 70%.
- **Unlock system:** Intermediate unlocks after Beginner is passed. Advanced unlocks after Intermediate is passed.
- **Saved notes:** Students can write and save lesson notes.
- **Quiz history:** The app remembers recent quiz attempts.
- **Rank:** The navbar shows a rank like Novice, Apprentice, Performer, Virtuoso, or Maestro.
- **Metronome:** A practice metronome is always available after login.

3. Project Folder Map

```
src/
├── main.jsx                Starts the React app
├── App.jsx                 Controls routes and page protection
├── App.css                 Main styling and layout
├── index.css               Small global CSS setup
├── components/
│   ├── Navbar.jsx          Top menu, rank, username, logout
│   ├── Metronome.jsx       Practice BPM tool
│   └── Components.jsx      Small reusable UI pieces
├── data/
│   └── data.js             All instruments, lessons, quizzes, resources
├── pages/
│   ├── LoginPage.jsx       Login, register, demo login
│   ├── HomePage.jsx        Instrument selection
│   ├── DashboardPage.jsx   Lesson list and tier buttons
│   ├── LessonPage.jsx      Video, notes, diagram, quiz button
│   ├── QuizPage.jsx        Questions, score, review, history
│   ├── ProgressPage.jsx    Progress bars for all instruments
│   └── ResourcesPage.jsx   Extra learning links
└── utils/
    ├── auth.js             Login/register/localStorage logic
    └── progress.js         Progress, notes, quiz history, unlock logic
```

Outside `src`, the important files are `package.json` for dependencies and scripts, `vite.config.js` for build settings, and `.github/workflows/deploy.yml` for GitHub Pages deployment.

4. How The App Starts

The app starts in a simple chain:

index.html → main.jsx → App.jsx → page route → page component

1. `index.html` gives the browser a root div.
2. `main.jsx` tells React to render the app inside that root div.
3. `App.jsx` wraps the app in `HashRouter`.
4. `App.jsx` checks if the user is logged in.
5. If logged in, the user can see the main pages. If not, the app sends them to Login.

Easy way to explain: `main.jsx` turns React on. `App.jsx` decides what page to show.

5. Navigation / Routing

Routing means moving between pages without reloading the whole website. Crescendo uses `react-router-dom`.

Route	Page	What it shows
`/login`	LoginPage	Login, register, demo account.
`/`	HomePage	Instrument grid and progress summary.
`/dashboard/:instrument`	DashboardPage	Lessons for a selected instrument.
`/lesson/:instrument/:tier/:lessonId`	LessonPage	One exact lesson.
`/quiz/:instrument/:tier/:lessonId`	QuizPage	Quiz for one exact lesson.
`/progress`	ProgressPage	Progress bars.
`/resources`	ResourcesPage	Extra links.

The parts with a colon, like `:instrument`, are dynamic. That means one page can show different content depending on the URL. For example, `/dashboard/guitar` and `/dashboard/piano` both use `DashboardPage`, but the data shown is different.

Why HashRouter? GitHub Pages can break on normal routes. HashRouter makes URLs like `#/login`, which keeps the app from showing 404 errors.

6. Login and Register Explained

The login system is handled by `LoginPage.jsx` and `auth.js`.

What LoginPage does

- Shows the form.
- Stores what the user types using `useState`.
- Checks if email/password are valid.
- Shows password requirements during registration.
- Calls `login()`, `register()`, or `loginDemo()`.

What auth.js does

- Saves accounts in `crescendo_accounts`.
- Saves the current logged-in user in `crescendo_user`.
- Checks duplicate email during registration.
- Removes the current user during logout.

```
Register flow:
User fills form → LoginPage validates → auth.register()
→ account saved in localStorage → user is logged in → go to home

Login flow:
User enters email/password → auth.login()
→ matching account found → session saved → go to home

Logout flow:
Navbar logout button → auth.logout()
→ remove current user → go back to login
```

Important note: This is good for a frontend class project, but not for a real app. Real passwords should not be stored in localStorage.

7. State Explained Simply

State is data that can change while the app is running. When state changes, React updates the screen.

File	State	Simple meaning
LoginPage	`fields`, `errors`, `mode`	What the user typed, validation messages, login/register mode.
DashboardPage	`activeTier`	Which tier button is selected.
LessonPage	`notes`, `markedForReview`	Student notes and review mark.
QuizPage	`answers`, `result`, `history`	Selected answers, final score, old quiz attempts.
Metronome	`bpm`, `playing`	Tempo number and whether sound is playing.

Easy explanation: If the user clicks something or types something and the screen changes, that is usually React state working.

8. Data and Lessons

All lesson content is stored in `src/data/data.js`. This file is basically the app's built-in mini database for lessons.

It contains:

- `INSTRUMENTS`: the 6 instruments shown on the homepage.
- `TIERS`: beginner, intermediate, advanced.
- `TIER_LABELS`: display names for tiers.
- `INSTRUMENT_DATA`: all lessons, videos, explanations, diagrams, and quizzes.
- `RESOURCES`: outside learning links.

Simple count: 6 instruments × 3 tiers × 3 lessons = 54 lessons. Each lesson has 4 quiz questions, so the app has 216 quiz questions.

9. Progress System

Progress is handled in `src/utils/progress.js`. This is one of the most important files because it decides what is complete, what is locked, and what rank the student has.

Function	What it does
`loadProgress()`	Reads saved progress from localStorage.
`saveProgress()`	Saves progress back to localStorage.
`markLessonComplete()`	Marks a lesson as passed after quiz success.
`isTierUnlocked()`	Checks if a tier can be opened.
`saveLessonNotes()`	Saves lesson notes.
`recordQuizAttempt()`	Saves quiz score history.
`getStudentRank()`	Returns the rank shown in the navbar.

```
Unlock rule:
Beginner = always open
Intermediate = open only if all Beginner lessons are passed
Advanced = open only if all Intermediate lessons are passed
```

Easy explanation: `progress.js` is the brain of the learning system.

10. Quiz System

The quiz system is in `QuizPage.jsx`. It loads the correct quiz based on the URL, stores selected answers, checks the score, shows explanations, and saves history.

```
Quiz flow:
User selects answers
→ Submit Examination
→ app compares selected answers to correct answers
→ score is calculated
→ if score >= 70%, lesson is marked complete
→ quiz attempt is saved to history
→ result screen appears
```

The passing score is stored as `PASSING_SCORE = 70`. If the student passes, `markLessonComplete()` is called.

Easy explanation: QuizPage checks the answers. progress.js saves the result.

11. Lesson Page

`LessonPage.jsx` is where students study. It uses the URL to find the correct lesson from `data.js`.

It shows:

- YouTube lesson video.
- Technical explanation paragraphs.
- Diagram image.
- Lesson notes textarea.
- Save notes button.
- Mark for review button.
- Previous and next lesson buttons.
- Start quiz button.

Easy explanation: LessonPage is the study room. QuizPage is the exam room.

12. Main Components

Component	Simple explanation
`Navbar`	The top menu. It shows Home, Progress, Resources, username, rank, and Logout.
`Metronome`	The practice tool at the bottom. It lets students change BPM and start/stop sound.
`ProgressBar`	A reusable progress meter used in dashboard/progress pages.
`TierBadge`	A small label showing the current tier.

These components make the app easier to maintain because the same UI idea can be reused in many places.

13. Backend Status

The live React app does not currently use a backend. It saves accounts and progress in localStorage.

There is a backend starter file at `server/server.js`. It has API routes for register, login, and progress. It also hashes passwords better than the frontend version. However, the frontend does not call it yet.

How to explain this: “Right now the project is frontend-only for demonstration. A backend scaffold exists, but it is future work for cross-device saving and better security.”

14. What Is Not In The Project

Feature	Status
Firebase / Supabase	Not used.
Realtime chat	Not used.
AI features	Not used.
Calendar or task system	Not used.
Pomodoro timer	Not used.
Pet system	Not used.
Gamification	Partly used through ranks, progress bars, tier locks, quiz history, and passing feedback.

15. Files That Are Safe Or Risky To Edit

File	Risk	Why
`data.js`	Risky	Lesson IDs connect to routes, notes, quiz history, and progress. Changing IDs can break saved data.
`progress.js`	Risky	Controls completion, tier unlocking, notes, rank, and quiz history.
`auth.js`	Risky	Controls login, register, demo user, and logout.
`App.jsx`	Risky	Controls all routes and page protection.
`vite.config.js`	Careful	The base path is needed for GitHub Pages.
`ResourcesPage.jsx`	Safer	It only shows a list of links.
`Components.jsx`	Safer	Small reusable components, easier to refactor.

16. Issues And Improvements

Things that work but can be improved

- **Plain localStorage passwords:** okay for demo, not okay for production.
- **Large data.js:** it has almost 2000 lines, so it should later be split by instrument.
- **Large App.css:** styling works, but the file is very long and should later be split into smaller CSS files.
- **Backend not connected:** the optional backend exists but the frontend does not use it yet.
- **No automated tests:** tests would help make sure quiz scoring and tier unlocking do not break.
- **External media:** YouTube and Unsplash links depend on outside services.

Best future improvements

1. Add real backend login and progress sync.
2. Split `data.js` by instrument.
3. Split `App.css` by purpose.
4. Add tests for progress and quiz logic.
5. Add a React Context provider for user/progress state.

17. Presentation Q&A Cheat Sheet

Question	Simple answer
What is Crescendo?	A React music-learning app with lessons, quizzes, notes, progress, ranks, and a metronome.
What framework did you use?	React with Vite.
Where are the routes?	In `src/App.jsx`.
Why use HashRouter?	So GitHub Pages does not show 404 errors on login/logout or refresh.
Where is state used?	Login form, dashboard tier, lesson notes, quiz answers/results, and metronome BPM.
Where is the lesson data?	In `src/data/data.js`.
How many lessons?	54 lessons total.
How does unlocking work?	The app checks if all lessons in the previous tier are passed.
How is progress saved?	In localStorage through `progress.js`.
Does it use a backend?	Not in the live app. There is only a backend scaffold for future use.
What is the biggest weakness?	localStorage auth is not secure for production.

18. Final Easy Mental Model

If you only remember one thing, remember this:

> **data.js gives the lessons → pages show them → progress.js saves what the student does → App.jsx controls where the user goes**

So the project has four main ideas:

1. **Data:** all lessons and quiz questions.
2. **Pages:** screens the user sees.
3. **Utilities:** logic for auth and progress.
4. **Router:** decides which page to show.

That is the core of Crescendo.